LM 기반 영어 명령 해석과 자동차 제어

프로젝트 계획서 · 2026.10.07 · 발표 2026.10.20

영어 단어·짧은 문장을 메인 Arduino Uno R3가 받아 두 계산 Uno에 보낸다. 두 보드의 신경망 계산 결과를 합친 뒤, 메인 D2/D3 software UART로 제어 Uno에 명령을 전달한다. 제어 Uno는 전진·후진, 조명, 와이퍼 서보를 담당한다.

이번 계획의 목표

LM은 프로젝트의 언어 모델 주제를 뜻한다. 실제 초기 알고리즘은 짧은 영어 문장을 8개 의도로 분류하는 작은 학습형 신경망이다. 자유 문장을 생성하는 LLM·ChatGPT·SmolLM을 보드에 올리는 계획은 아니다. 한국어는 지원하지 않는다.

구분	계획	현재 상태
언어 해석	새 8의도 모델 학습·2보드 분할 추론	구현·학습 예정
자동차 제어	모터 1개 시험대 → 차체, LED, 와이퍼 서보	부품 미정·실물 미검증
통신	메인에서 UART·I2C·SPI 실제 전송 중첩	배선·파형 검증 예정

과제 원문

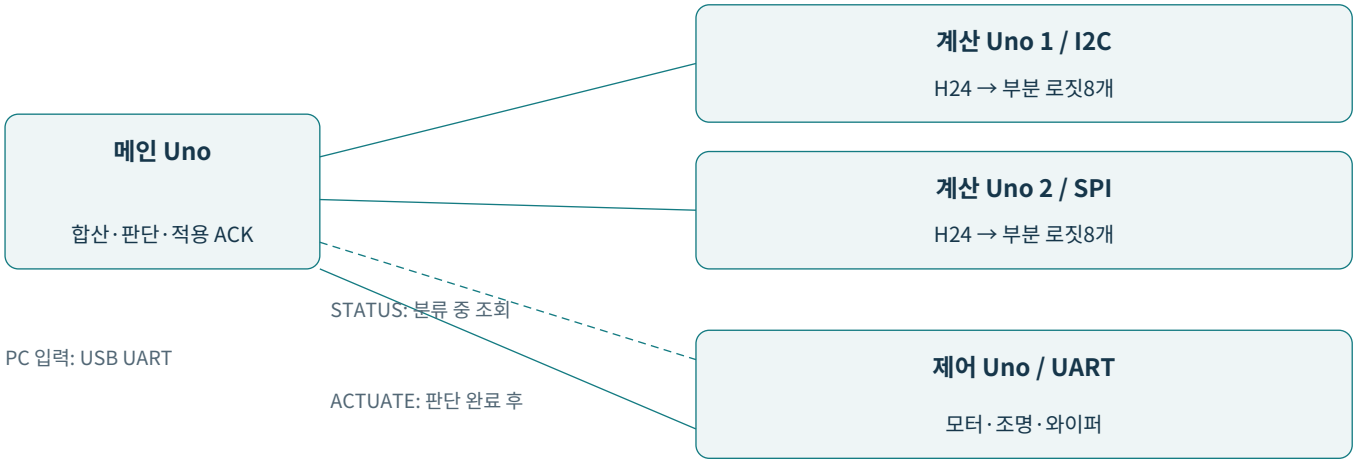
“Using 4 arduinos, create a cooperating application that utilize all 3 communication methods. (UART, SPI, I2C) At least 1 arduino uno should run all three protocols at the same time. The reports can be the same for the teams. However, individual reports should contribution (yes/no) for each of the members (excluding yourself) on the first page. Prepare for presentation on Oct 20. The presentation must be under 5 minutes.”

성공의 정의

네 보드가 영어 명령의 해석과 실제 구동을 협력 수행한다. 메인은 I2C·SPI 추론 전송 중 UART로 제어 보드의 현재 출력상태를 조회하여 세 전송의 공통 중첩 구간을 기록한다. 새 구동 명령은 분류 결과가 확정된 뒤에만 보낸다. 출력은 일치하는 적용 ACK를 받은 뒤 APPLIED로 표시한다.

이 문서는 계획서이며 실제 자동차 제어의 완료 보고서가 아니다. 이름·팀 정보는 포함하지 않는다. 최종 개인 보고서 첫 페이지에는 본인을 제외한 각 팀원의 기여 여부 Yes/No를 작성해야 한다.

시스템 흐름: 두 계산 보드와 한 제어 보드



실선: 계산·최종 제어 | 점선: 동시 상태 조회 | 부품·실물 검증 예정

입력부터 물리 동작까지

영어 입력 → 길이·어휘 검사와 정규화 → I2C/SPI 부분 로짓 → 메인 합산·의도 판단 → UART ACTUATE → 제어 Uno의 상태 적용 → ACK → 사용자에게 APPLIED 표시 순서이다. 신경망의 자유 텍스트를 GPIO 신호로 해석하지 않고, 허용된 의도 enum만 전달한다.

점선 STATUS 경로는 분류 중 상태 조회용이다. 실선 ACTUATE 경로는 두 계산 보드의 결과를 받은 이후의 동작 명령이다. 계산 중에 전진·후진 결과를 추측하여 먼저 구동하지 않는다.

구성	보드·모델 배치	과제 충족
계산 Uno 1대	메인 + H48 전체 계산 + 제어 = 3대	개발·비교 기준. 4대·3프로토콜 조건 미충족
계산 Uno 2대	메인 + I2C H24 + SPI H24 + UART 제어 = 4대	최종 목표. 동일 H48 모델 분할

역할의 경계

PC는 학습·export·업로드·검증 및 입력/표시에 사용한다. 실행 중 영어 의도의 추론 산술과 제어는 Uno에서 수행한다. 두 계산 보드가 세 번째 UART 계산 보드를 요구하는 구조가 아니다. UART는 자동차 제어 보드와 연결한다.

메인은 USB 하드웨어 UART D0/D1로 입력을 받고, 별도 software UART D2 RX/D3 TX로 제어 보드와 통신한다. 계산 보드의 I2C와 SPI는 각자의 부분 점수를 반환한다. 제어 보드는 GPIO/PWM·서보 setpoint·소프트웨어 상태를 ACK/STATUS로 보고한다. APPLIED는 출력 적용을 뜻하며 센서로 물리 동작을 확인했다는 뜻은 아니다. 실제 이동·회전은 영상·관찰로 따로 검증하고 센서는 향후 선택 사항이다.

영어 의도 데이터와 학습형 모델

같은 의도, 다른 영어 표현

최대48 ASCII 바이트의 한 줄을 받으며 초과 입력을 자르지 않고 거절한다. 소문자 변환·공백 압축·끝의 .!? 제거는 명시적인 전처리이다. 의미가 있는 not 같은 단어는 삭제하지 않는다. apostrophe 축약형은 초기 버전에서 거절한다. 모델 어휘는 pad/newline0, space1, a~z2~27, period28의29개이다.

ID / 의도	학습할 유사 표현 예시 - 아직 학습 예정
0 FORWARD	go forward / move ahead / drive forward / please move forward / can you move ahead
1 REVERSE	move backward / go backwards / reverse / back up / please move back
2 STOP	stop / stop moving / halt / stop the car
3 LIGHT_ON	switch on the lights / turn the headlights on / lights on please
4 LIGHT_OFF	switch off the lights / turn the headlights off / lights off please
5 WIPER_ON	start the wiper / turn on the wipers / wipe the windshield / start wiping
6 WIPER_OFF	stop the wiper / turn off the wipers / stop wiping
7 UNKNOWN	do not move forward / do not turn lights on / unrelated or compound text

문장군을 나눈 학습·평가 - 구현 예정

동작7개 의도별25~40문장(175~280)과 UNKNOWN50~80문장으로225~360개를 목표로 한다. 초기 약160개로 시작해 확대한다. 단어·어순·공손한 접두어를 다양하게 학습하며 정확한 문구 사전으로 전체 명령을 처리하지 않는다. 알려진 동의어와 새 어순·please 표현을 시험하되 완전히 처음 보는 동의어까지 이해한다는 보장은 없다.

같은 문장·template family는 같은 그룹으로 묶어 train/validation/test를60/20/20 후보로 분리한다. 학습문장의 단순 변형이 test로 새지 않게 한다. validation은 임계값 교정에 쓰고 frozen test는 손대지 않는다. PC NumPy 고정seed 학습과 데이터·라벨·가중치 버전을 기록한다. 기존40단어·30문장 corpus는 새 데이터가 아니다.

판단·거절·빠른 정지

8의도 점수의 confidence0.8·margin0.2는 검증 데이터로 교정할 초기 후보이다. softmax 값이 안전성이나 실제 확률을 보장하지 않는다. UNKNOWN·낮은 확신·부정·복합 명령은 주행STOP, 조명은 유지한다. “go forward and turn lights on”은 초기 한 입력 한 동작 원칙에서 거절한다.

정확한 stop은 신경망을 우회하는 빠른 정지 경로이다. 필요 시 not/and 형태를 먼저 거절하는 보수적 입력 gate를 추가하고 보고서에 공개한다. 의도 분류 자체는 학습형 모델로 수행하며, 부정문을 전진으로 바꾸지 않는다.

신경망 분할과 export: 새 모델을 재학습

전체 모델과 텐서

고정 48문자 위치 × 임베딩 4차원 = 192개 입력 → H48 ReLU → 8개 의도 로짓 구조이다. 새 8의도 출력층과 데이터로 전체 모델을 다시 학습한다. 기존 다음 문자 예측용 48뉴런 가중치는 자동차 제어용으로 학습된 것이 아니다.

배열	전체 모델	I2C / SPI 분할
e	[29,4]	각 보드에 전체 복제
w / b	[192,48] / [48]	w[:,0:24], b[0:24] / w[:,24:48], b[24:48]
o	[48,8]	o[0:24,:] / o[24:48,:]
c / scale	c[8], se/sw/so	c는 메인에만, 기존 텐서 scale 유지

각 계산 보드는 pk = ReLU(flatten(E[input]) · Wk + bk) · Ok의 8개 float 부분 로짓, 총 32 B를 돌려준다. 메인은 c + pI2C + pSPI를 계산한다. 출력 bias는 한 번만 더하고 계산 보드별 최상위 의도만 보내지 않는다.

모델 데이터 예산 - 아직 컴파일 미측정

모드	저장 데이터 산식	계획 값
1계산 구성(연산+메인)	계산 e116+w9216+o384+b192 =9908; 메인 c32	합계9,940 B, scale12 B 제외
2 계산 보드 각 shard	e116 + w4608 + o192 + hidden bias96	각 5,012 B, scale 12 B 제외
2보드 합계 + 메인 bias	5012×2 + c32	10,056 B, 복제 scale 제외

위 값은 배열 데이터 계산이며 코드·통신 버퍼·메타데이터·실행 스택을 포함한 측정 결과가 아니다. PROGMEM에 가중치·bias를 두고 int8은 pgm_read_byte, float는 pgm_read_float로 읽는다. 각 보드의 빌드와 스택 high-water로 여유를 확인한다.

export_intent_split.py의 예정 절차

이미 양자화된 e/w/o와 float b/c를 읽고 숨은 뉴런 범위 [0,24), [24,48)를 자른다. 원래 se/sw/so를 유지하여 shard마다 재양자화하지 않는다. model_i2c.h, model_spi.h, main_bias.h에 모델·어휘·라벨 ID와 차원·담당 범위를 넣는다. 메인은 0~47이 중복·누락 없이 덮이는지 검사한다.

1계산 모드는 시작 시 선택한 별도 전체 모델 구성이다. 2계산 모드에서 한 보드가 응답하지 않으면 전체 모델로 몰래 fallback하지 않고 오류·STOP으로 처리한다.

배선과 전원: 보드별 핀·타이머 분리

메인과 통신 핀 [1]

링크	메인 Uno	상대 Uno	계획 설정
USB 입력	D0 RX / D1 TX	PC 시리얼 모니터	9600 baud
제어 UART	D2 RX / D3 TX	제어 D1 TX / D0 RX, 교차 연결	9600, 8N1
계산 I2C	A4 SDA / A5 SCL	같은 핀, 주소 0x12	100 kHz·pull-up
계산 SPI	D10 SS, D11 MOSI, D12 MISO, D13 SCK	같은 핀, slave D10 INPUT	Mode 0, MSB first, 125 kHz

메인 D10은 OUTPUT이다. 제어 보드는 UART 업로드 시 D0/D1 외부 연결을 분리하고, 정상 동작 중 USB 시리얼 모니터를 열지 않는다. 메인의 Timer1은 software UART용으로 예약한다. SPI D13은 SCK이므로 활동 LED로 쓰지 않는다.

제어 Uno의 출력 핀 - 부품 선정 후 확정

기능	제어 Uno 핀	연결·제약
모터 1개 H-bridge	D4 IN1, D7 IN2, D5 PWM enable	모터를 GPIO에 직접 연결하지 않음
조명	D8	LED는 220~330Ω 저항, 큰 조명은 별도 드라이버
와이퍼 서보	D9 signal	Servo는 Uno Timer1 사용; D9/D10 analogWrite 제한 [6]
선택적 로컬 정지 버튼	D2 INPUT_PULLUP	주행·와이퍼 정지, 메인 D2와는 다른 보드

Servo 타이머는 제어 보드에 있고 software UART 타이머는 메인에 있다. 제어 UART는 하드웨어 UART를 사용한다. 모터 PWM D5는 Timer0로 유지한다. 두 모터 차체는 추가 핀·드라이버 예산을 다시 잡고, 먼저 모터 1개 시험대로 전진/후진을 확인한다.

전원과 부품의 결정 사항 [7, 8]

모터 전압·stall current에 맞는 H-bridge를 선정한다. TB6612/L298N은 검토 예시이며 현재 선택된 부품은 없다. TB6612를 선택하면 STBY용 D6 또는 상시 enable 조건을 추가로 확정한다. 보호 회로와 방향 truth-table은 선택한 드라이버 데이터시트로 확인한다.

모터·서보는 적절한 외부 전원/레귤레이터와 드라이버로 공급하고 Uno와 공통 GND를 연결한다. 모터를 Uno GPIO/5 V에 직접 공급하지 않는다. 독립 USB 전원의 5 V 레일을 서로 묶지 않는다. 서보 각도·구동 전압·기구 간섭은 실물 부품 결정 후 확인한다.

공통 32바이트 패킷과 작업 상태

프레임 규격 - 구현 예정

바이트	필드	규칙
0 / 1 / 2	SOF / version / op	0xA5 / 1 / 허용된 명령 코드
3~4 / 5 / 6	id / seq / payloadLen	uint16 little-endian / 청크 번호 / 유효 길이
7~30	payload[24]	뒤쪽 0 padding
31	CRC8	poly 0x07, init/xorout 0, non-reflected; 앞31 B 검사

계산 op는 LOAD_INPUT, START, STATUS, GET_RESULT, RESULT, ACK, ERROR이다. 입력 48 B는 24+24 B, 결과 32 B는 24+8 B로 나눈다. float는 sizeof(float)==4·IEEE754 호환성을 확인하고 memcpy로 little-endian binary 직렬화한다. UART 제어 op는 ACTUATE, ACK, STATUS, HEARTBEAT, STOP이며 같은 envelope를 사용한다.

ID와 중복 방지

계산 job ID와 구동 command ID는 op별 의미를 구분한다. 계산 job ID 재사용 전에는 진행 중 작업을 종료하고 링크를 drain한 뒤 두 계산 보드를 reset/handshake하여 옛 결과가 남지 않게 한다. 시작 STATUS로 모델·어휘·라벨 ID·차원·shard 범위를 확인한다. 보드별2-bit chunk mask로 중복 RESULT를 무시하고 2청크 완료 후 한 번만 합산한다.

UART 제어 payload는 session/epoch uint32 4 B + intent1 B + duration uint16 2 B =7 B를 기본으로 하며 필요한 출력상태·버전도24 B 안에 넣는다. 제어 보드는 모든 ACTUATE/STOP/HEARTBEAT/STATUS에서 현재 session과 command ID를 확인한다. ACK는 session·command ID·적용출력상태를 되돌린다. 새 세션은 정지 상태의 handshake로 시작하고 캐시·pending 명령을 모두 무효화하며 재개하지 않는다. 같은 ID는 캐시 ACK만 반환하여 구동·duration을 재시작하지 않는다. 잘못된 op·intent·길이·CRC는 실행하지 않는다. session prefix는 UART 제어에만 적용하며 계산 입력48 B의24+24 분할은 유지한다.

링크별 송수신

링크	처리 방식
I2C	주소 0x12, Wire 최대32 B. onReceive는 버퍼·seq만 갱신, onRequest는 캐시된 frame만 반환 [2].
SPI	SS 아래 요청 → 응답 준비 확인 → 별도 read에서 dummy bytes. SPDR preload·첫 byte pipeline·slave ISR용 간격을 검증한다.
UART	메인 timer UART·제어 hardware UART, 유효 용량64 B 이상 ring buffer. SOF 재탐색·CRC·ACK 흐름 제어.

계산 FSM은 IDLE → RECEIVE → COMPUTE → RESULT_READY이다. 신경망은 main loop에서 계산하고 ISR/콜백은 짧은 버퍼 입출력만 수행한다. 통신 timeout과 분류 작업 deadline 2~10초 후보는 별도로 교정한다. 같은 ID의 재시도 최대2회 이후 명시적 실패로 종료한다. 실패·UNKNOWN은 주행 STOP이며 기존 동작을 자동 재개하지 않는다.

메인: 계산 결과 이후에만 동작 명령

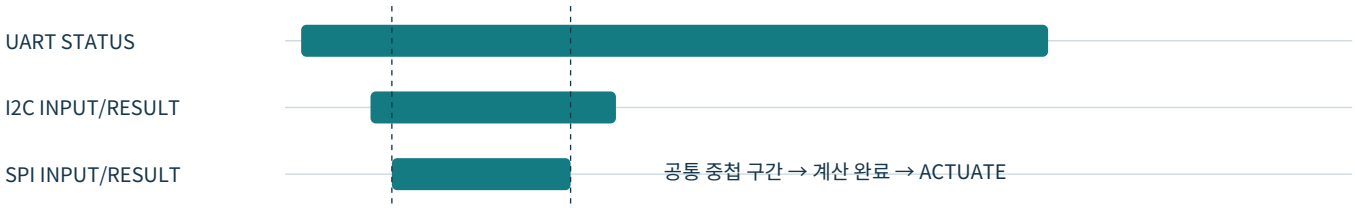
분류·제어의 의사코드 - 구현 예정

```
input = read_line(maximum 48 ASCII bytes)
validate_and_normalize(input)
if input == "stop": enqueue_priority_STOP(); wait_matching_ACK()
else:
    begin job(modelId, immutable input, new jobId)
    totals = output_bias; receivedMask = 0
    dispatch LOAD_INPUT + START to I2C and SPI
    while receivedMask != 0b11:
        poll_I2C(); poll_SPI(); poll_UART_status(same jobId)
        if complete matching partial8 and not already counted:
            totals += partial8; set worker bit
        if timeout/model mismatch: send_STOP(); fail job
    intent = argmax(totals); evaluate confidence/margin
    if UNKNOWN or rejected: intent = STOP
    send_UART_ACTUATE(intent, new commandId, duration)
    if matching ACK reports applied state: print APPLIED
    else: print ERROR; do not claim actuation succeeded
```

같은 job/model ID의 두 계산 결과를 모두 받아야 한다. context[48]은 작업 중 불변이며 totals[8] float와 청크 상태를 고정 버퍼로 둔다. 한 보드의 최상위 결과만 채택하지 않는다. 받은8개 float가 모두 finite인지 검사하고 NaN/Inf면 ERROR+STOP으로 처리한다. 분류 완료 전에 새 모터 동작을 보내지 않는다.

세 실제 전송의 공통 구간 확보

계획 타임라인 · 실측 아님 · 같은 분류 job의 실제 상태/모델 전송



STATUS는 구동하지 않음 · 새 ACTUATE는 두 점수 수신 이후

추론의 입력/결과 거래 중 UART STATUS 또는 HEARTBEAT으로 제어 Uno의 현재 출력상태를 확인한다. 이 거래에는 같은 분류 job tag를 넣고, I2C/SPI 실제 모델 전송과 세 가지가 겹치는 하나의 공통 시간 구간을 기록한다. STATUS/HEARTBEAT은 모터·조명·와이퍼를 바꾸거나 drive lease를 연장하지 않는다.

최종 ACTUATE는 계산 완료 뒤의 별도 UART 거래이다. 동시성 증거를 만들기 위해 아직 분류되지 않은 방향을 보내지 않는다. 단순한 별도 ping이나 계산만 겹친 로그는 과제 통과 증거가 아니다.

비차단 엔진과 제어 보드의 동작

메인 통신 엔진 - 구현 예정

엔진	초기 설정·검증
software UART	Timer1 /8, OCR1A=207: 약104 µs bit·9615 baud, 약0.16% 오차. D3 PORTD TX, D2 INT0 시작 검출 ·compare B 중앙 RX sampling.
I2C / TWI	16 MHz, TWBR=72, TWPS=0: 100 kHz. ISR/state machine으로 start/address/data/stop 진행 [5].
SPI	SPE·MSTR, SPR1:SPR0=11, SPI2X=0: /128 =125 kHz. SPIF poll·바이트 간격, Mode0·MSB first [4].

전체 프레임 동안 cli()를 유지하는 기본 SoftwareSerial TX는 사용하지 않는다 [3]. blocking Wire.endTransmission/전체 SPI 전송 대신 엔진별 짧은 단계로 loop에 복귀한다. Timer1 충돌·RX 위상·ISR 지터·유실 바이트를 파형으로 검증한다. 레지스터 설정만으로 동시성 달성을 주장하지 않는다.

제어 Uno: 독립적인 비차단 FSM

기능	동작 계획
전진 / 후진	낮은 PWM·유한 duration. 반대 방향 요청 시 PWM0 → 선택한 driver의 neutral → 약100 ms deadtime 후보 → 새 방향.
조명	ON/OFF를 독립 latch. 다른 정상 명령이 조명을 초기화하지 않음.
와이퍼	millis 기반 20~30 ms마다 2~5° 이동, 45~135° 초기 후보. 기구 간섭 확인 후 조정.
WIPER_OFF	왕복을 중지. 필요 시45°로 천천히 park하는 옵션을 설정.
STOP / 로컬 ESTOP	STOP은 주행 모터만 정지. 로컬 버튼은 모터+와이퍼 정지, 조명 상태 유지.

부팅 시 모터 정지·조명 OFF·와이퍼 OFF이다. delay 왕복 루프를 사용하지 않고 UART 수신과 STOP 처리를 계속한다. 정확한 stop 입력은 현재 UART 프레임을 마친 다음 우선 전송하며 프레임 중간을 끊지 않는다. 로컬 정지 버튼은 메인·분류 실패와 독립적으로 동작한다.

구동 시간의 제한과 오류 후 상태

초기 데모 DRIVE는 500 ms duration, 최대1000 ms cap 후보로 자동 정지한다. 실제 속도·시간은 시험대에서 교정한다. 제어 보드의 자체 타이머가 메인 정지·분류 지연과 무관하게 만료를 처리한다. 같은 명령 ID의 재전송이나 500 ms HEARTBEAT은 duration을 갱신하지 않는다.

연속 구동을 추가한다면 현재 명시적으로 승인된 drive intent에 한해서만 새 command ID로 갱신한다. UNKNOWN·timeout·새 분류 실패 뒤에는 자동 갱신하지 않는다. 잘못된 패킷은 실행하지 않으며, 오류 STOP 정책과 만료 시 주행 정지를 시험한다.

구현 단계와 예정 파일 구조

모듈의 책임 - 모두 새 구현 예정

예정 경로	책임
firmware/main/main.ino	입력 정규화·합산·거절·ACK·비차단3링크
firmware/compute_i2c/compute_i2c.ino	I2C frame·H24 부분 추론·결과 캐시
firmware/compute_spi/compute_spi.ino	SPI pipeline·H24 부분 추론·결과 캐시
firmware/actuator_uart/actuator_uart.ino	hardware UART·모터/LED/Servo FSM·만료
firmware/shared/protocol.h / neural_shard.h	ID·CRC·enum / PROGMEM 부분 계산
host/train_intent.py / export_intent_split.py	8라벨 모델 학습·2분할 export
host/test_intent_distributed.py	1/2계산 수치 비교·거절·학습 품질

단계별 통과 조건

단계	작업	저장할 증거
A 데이터·모델	영어8의도 데이터·holdout → H48 학습	데이터 split·model ID·오분류표
B 호스트 분할	H24+H24 로짓 합·선택 의도 비교	50입력 CSV·오차·라벨 ID
C 세 링크	32 B ACK/CRC 왕복·STATUS 현재 출력상태	UART/I2C/SPI 로그·중첩 trace
D 제어 시험대	모터1개·LED·서보 개별 → ACK 연결	핀·driver 설정·동작 영상
E 통합·오류	영어→의도→UART→실물·누락/CRC/만료	APPLIED 로그·복구·정지 확인
F 비교·발표	1계산/2계산 동일모델 지연·메모리	p50/p95·4빌드·스택·발표 영상

기존 tools/test.sh와 tools/build.sh는 문장 생성 기준 코드를 검증한다. 새 자동차 제어 파일이 구현되면 각 스케치에 arduino-cli compile --fqbn arduino:avr:uno를 적용한다. 예정 명령 python3 host/train_intent.py → python3 host/export_intent_split.py → python3 host/test_intent_distributed.py는 현재 존재하는 실행 도구가 아니다.

부품이 정해지면 드라이버 truth-table·PWM·전원·서보 기구 한계를 먼저 확정한다. 두 모터 차체 확장은 모터1개 시험대·통신·정지 시험이 통과한 이후에 진행한다.

시험 계획: 의도·통신·동작을 각각 확인

정확도와 수치 동등성

시험	예정 통과 기준·관찰 항목
학습 품질	문장군 분리 test set, macro-F1·라벨별 recall·confusion matrix·UNKNOWN 거절률
1계산 vs2계산	고정50입력, 로짓 abs/rel 1e-4 이내·선택 의도 일치
영어 경계	대문자·공백·48/49 B·비ASCII·지원 밖 문자·긴 줄 복구
부정·복합	go not forward, 두 동작 요청을 동작 거절로 분류
교정	confidence/margin 교정, test 위험한 동작0건 목표(측정 결과 아님)

오류·구동·동시성

시험	확인할 실제 행동
CRC / 미허용 enum / 옛 session	동작하지 않음·ERROR 또는 정책상 STOP
중복 ACTUATE	캐시 ACK, 모터 duration 재시작 없음
계산 보드 누락 / 모델 불일치	한 보드 fallback 없음·주행 STOP·다음 입력 복구
분류 지연 / 메인 종료	제어 Uno의 DRIVE 만료·자동 정지, 재개 없음
정지·방향 전환	STOP·로컬 버튼·neutral/deadtime·UART 계속 수신
ACK / 물리 동작	APPLIED는 출력 적용 ACK, 실제 이동·서보 왕복은 영상·관찰로 별도 검증
3프로토콜 동시성	UART STATUS + 실제 I2C/SPI 추론 전송 공통 구간

로직 분석기는 UART TX, I2C SDA/SCL, SPI SCK/MOSI/SS 6채널을 기록한다. 필요 시 UART RX·SPI MISO를 추가하여8채널로 확장한다. job tag와 디코딩된 frame을 trace CSV/VCD·스크린샷으로 저장한다. heartbeat와 STATUS는 제어 보드의 출력상태를 확인하는 실제 거래이며 새 actuation을 실행하지 않는다.

아직 측정되지 않은 비교표

1계산(3보드 개발 구성)과 2계산(4보드 제출 구성) 모두 새 모델의 지연·Flash·SRAM은 미측정이다. 입력→판단, 판단→ACK, 입력→APPLIED의 p50/p95와 각 보드 메모리를 나눠 기록한다. 4대의 빌드와 런타임 스택 high-water를 확인하고 SRAM 여유512 B 이상을 잠정 목표로 둔다.

기존 문장 생성 빌드 Flash8614 B·전역SRAM462 B는 새 제어 모델이나 제어 펌웨어의 측정값이 아니다. 속도 개선·부품 동작·안전한 차체 운행을 미리 보장하지 않는다.

10월 20일 발표까지의 실행 계획

단계별 일정과 산출물

기간	작업	통과 산출물
10/7~9	영어 데이터·H48 모델·부품/시험대 결정	label·split·model ID·핀/전원 계획
10/10~12	I2C/SPI 부분 추론·UART 제어·ACK	50입력 비교·각 링크·개별 구동
10/13~15	4보드 통합·STATUS 동시통신·정지/만료	중첩 trace·영어→ACK→동작
10/16~18	오류·부정/복합·지연·메모리 시험	시험표·오분류·빌드·스택·영상
10/19	4분40초 리허설·자료·최종 보고서	타이머·영상·타인Yes/No 기여 표시
10/20	5분 미만 발표	실물 동작·추론/동시통신 증거·제한

발표 스토리보드: 총4분40초

시간	내용	화면·증거
0:00~0:35	영어 명령으로 자동차 기능 제어	문제·과제4대/3통신 요건
0:35~1:15	8의도 모델·1계산/2계산	데이터·분할·거절 정책
1:15~2:10	4보드·핀·결과 이후의 구동	구조도·ACTUATE/STATUS 구분
2:10~3:20	전진/후진·조명·와이퍼·STOP 시연	영어입력·적용출력 ACK·동시trace
3:20~4:05	동등성·오류·시간·메모리	실측 비교·만료/CRC·오분류
4:05~4:40	결론·제한·기여	달성 요건·부품/모델 범위·역할

현장 시연과 제출 자료

한 입력 한 동작으로 짧게 시연하고, 전진·후진은 교정된 낮은 속도와 유한 duration으로 시험한다. 영상은 현장 실패에 대비한 기록이며, 실제4보드 동작과 세 실제 전송 중첩을 확인하는 요건을 대체하지 않는다. 미달성 항목은 완료로 적지 않는다.

최종 개인 보고서 첫 페이지에는 본인을 제외한 각 팀원의 Yes/No 기여 표시를 추가한다. 발표·보고서에 실제 사용한 모델, 데이터, 펌웨어와 부품 정보를 기록한다. 새로운 제어 모델과 기존 문장 생성 데모를 구분한다.

참고 자료와 현재 증거의 경계

공식 기술 자료

[1] Arduino Uno R3 데이터시트: https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf

[2] ArduinoCore-avr Wire.h (32 B 버퍼): https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/Wire/src/Wire.h

[3] ArduinoCore-avr SoftwareSerial.cpp (TX/cli): https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/SoftwareSerial/src/SoftwareSerial.cpp

[4] ArduinoCore-avr SPI.h (clock divider): https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/SPI/src/SPI.h

[5] ArduinoCore-avr twi.c (TWBR 설정): https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/Wire/src/utility/twi.c

[6] Arduino Servo AVR 타이머 선택: https://github.com/arduino-libraries/Servo/blob/master/src/avr/ServoTimers.h

[7] Arduino 모터 제어 가이드: https://docs.arduino.cc/learn/electronics/transistor-motor-control/

[8] Arduino 서보 가이드: https://docs.arduino.cc/learn/electronics/servo-motors/

현재 로컬 프로젝트가 입증한 것

로컬 자료	현재 증거	새 자동차 제어에서의 의미
README.md / VERIFICATION.md	1보드 문자 생성 컴파일·호스트/mock 시험	빌드·검증 흐름 참고, 제어 성능 아님
model_metadata.json / model.npz	문맥16·H48·29문자 다음 문자 모델	새48문자·8의도 모델로 재학습 필요
tiny_uno/ / tools/	PROGMEM·고정 버퍼·학습/export 기반	제어 코드·driver·2분할은 앞으로 구현
docs/project-plan.md	이번 설계·규격·시험 기준	실물·추론·동시통신·ACK 모두 검증 예정

Arduino는 연결되지 않은 상태이며 새 모델·4보드 통신·모터·조명·서보 실물 검증은 없다. 부품도 아직 결정되지 않았다. 모델 데이터 예산·baud/clock·타이밍 후보는 설계값으로 표시했고, 시험 결과는 실제 측정 후 채운다.

선택할 H-bridge·모터·서보의 데이터시트는 부품 확정 후 추가한다. 현재 자료가 특정 부품의 전원·보호 회로·기계적 동작을 보증하는 것은 아니다.